

File Access Control

1. Basics

Linux (and Unix) enforce file access control using file permissions. Access control determines the rights a user (respectively a process, as any process runs with the privileges of a given user id) is granted on a specific file or directory. For example, a group of users may only have read permissions for a file while a specific user could have more rights, e.g. permission to write the file (= change the contents).

To enable this, the Linux file system maintains additional information on the file including the owner of a file, permissions for a group of users on that file and more. This is stored in a data structure which is part of the file meta-data (typically called an *inode*). Similar meta-data is maintained for folders (directories) – for example, to define who is permitted to list the contents of the directory, if someone can change their current working directory to this directory or may add new entries (= *links*) to the folder.

2. File Permissions

Each and every file has exactly two “ownership attributes”:

1. Owner
2. Group

File permissions can (only) be granted for these *three* categories of users:

1. Owner
2. Members of the file’s group
3. All others (who are not the owner and not member of the file’s group = “the world”)

For each of these user categories, the system stores information on three types of permissions for a file:

- i. Read the file,
- ii. Write the file (change its contents)
- iii. Execute the file (as a binary program or as shell script)

This applies to directories (folders) as well, but *read*, *write* and *execute* have a slightly different meaning:

- i. “Read” means: be allowed to list the directory’s *content* (what files and folders are in the directory?)
- ii. “Write” means: be allowed to create new files within this directory (= create new *links*), to delete entries in the directory or change the name of files in the folder
- iii. “Execute” means: it is *permitted to change into this directory* and the directory can be used as part of a path locating a particular file (one needs *execute permission on each path component* when accessing a file – this is different from Windows!)

The System checks for these permissions in above order (1. - 3.) and if there is a match for a specific user category, the matching permissions apply. Any other permissions granted “further down the line” have no relevance! This implies, for example that if a file is world-writable (the permissions for “all others” – this is category 3 – include the *write* permission), but permissions for the owner (category 1) are only *read*, then the owner can *not* change the contents of the file, even though the file is writable by anybody else. Note that this is very different from the way file permissions work in Windows!

Note: With this (simple) scheme, from above, there is another big difference to Windows NTFS file permissions: You can only provide permissions on a file to *one* user (the owner), to *one* group of users (the users that are member of the file’s group) and, as a “catch-all”, permissions for everybody else.

As a consequence, Linux users typically are members of multiple groups (to permit a reasonable granularity of rights).

3. Listing file permissions

The option **-l** (*long listing*) to the **ls** command (**ls -l**) can be used to display file permissions. To see the permissions for a directory (and not the permissions of the files listed), one would use the additional flag **-d** (e.g. **ls -ld MyFolder**).

Examples:

```
-rw-----
-rw-r--r--
-rwxr-x--
-rw-rw-rw-
drwxrwx---
drwxr-xr-x
```

The first character corresponds to the file type, e.g.:

- "-" regular file
- "d" directory
- "l" symbolic link

The nine subsequent characters have this meaning:

r	w	x	r	w	x	r	w	x
Owner			Group			Others		

- "r" stands for *read* permission
- "w" stands for *write* permission
- "x" stands for *eXecute* permission for files and, for directories = permission to "use" the directory (enter the directory and/or use the directory as a component of a path in order to access a file)

In the previous, last example line, "**drwxr-xr-x**" spells out to:

- "d" → It is a directory.
- "rwx" → The owner has read permission to the folder (= may list the contents), write permission (can create, rename and delete files) and may "use" the directory (= is permitted to enter the directory and access a file in this directory).
- "r-x" → All members of the file's group have read and "use" rights on the directory, so they may list the contents, enter the directory or use it to access files in this directory. They do *not* have write permission, meaning they cannot create new files, rename or delete files in this directory.
- "r-x" → All other user have read and "use" permissions as well.

4. Changing permissions with **chmod** (using symbolic notation)

The command to assign or change permissions is **chmod**, which is followed by several letters and then the

file name(s). The first letter specifies the category:

- "u" for the owner ("user")
- "g" group
- "o" for all others
- "a" is a short-hand for all three categories (owner, group and others)

The next letter:

- "+", to add the specified permissions to already existing permissions (if any)
- "-", to remove the specified permissions from already existing permissions (if any)
- "=", to set precisely the specified permissions, irrespective of any prior settings

Finally, the actual permissions are specified as follows:

Letter	Rights Granted
r	read
w	write
x	execute – <i>this has a special meaning for directories ("use")!</i>
s	setuid/setgid – <i>see explanation on SETUID and SETGID below!</i>
t	sticky bit – <i>see explanation on STICKY BIT below!</i>

Examples:

chmod u+rw foo → Allow the owner to read, write and execute the file **foo**.
chmod a+rw foo → Permit all users to read and write the file **foo**.
chmod o-w *.txt → Prevent all users (if owner and group also do not have write permission) from writing to the files with a file extensions of ".txt" in the current directory
chmod g-wx foo → Removes any existing *write* and *execute* rights for members of the file's group. If **foo** is a directory, these users can no longer enter the directory.
chmod u=rwx,g=rx,o= → Sets permissions to: All rights to the owner, only *read* and *execute* to the group and no permissions to all others

5. Changing permissions with chmod using octal numbers

As an alternative one may also use *octal numbers*. Permissions for each user category (owner, group, others) are summarized in a single octal number (0 to 7). Permission "r" corresponds to the octal number 4 (100), "w" to 2 (010) and "x" to (001) zugeordnet – so each permission triplet is read as binary number (0 ... permission is not set; 1 ... permission is set). Then the individual values are added up for each user category and converted into octal digits. The three octal digits are then concatenated – so the digits, read from left to right stand for: User permissions, group permissions and permissions for all others.

For example, an octal representation of 750 (equals binary pattern 111 101 000) grants all rights to the owner, *read* and *execute* to the group and no rights to everybody else (**rw-r--r--**).

Examples:

chmod 750 myprog → Sets permissions for program **myprog** to "**rw-r--r--**"
chmod 777 counter.dat → Sets permissions for file **counter.dat** to "**rw-rw-rw-**"
chmod 666 users.txt → Sets permissions for file **users.txt** to "**rw-rw-rw-**"

